



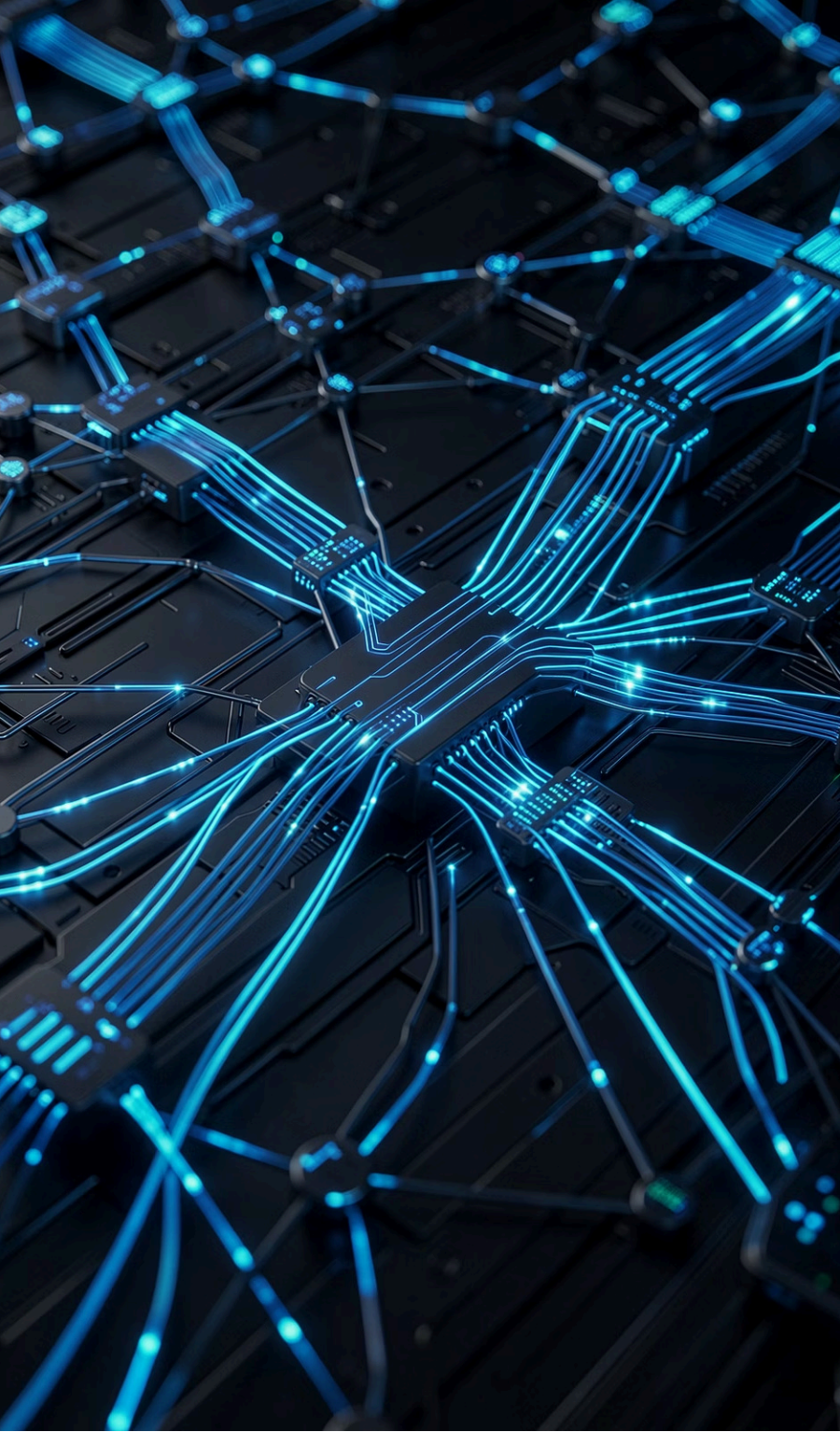
Database Fundamental

A practical guide to database design, architecture decisions, and production engineering practices — built for information security students stepping into backend development.

BACKEND ENGINEERING

DATABASE DESIGN

SYSTEM THINKING



Why Databases Matter

Applications generate critical data — users, assets, findings, scan results, logs. That data must survive restarts, crashes, and scale. Backend engineers own data integrity, consistency, availability, and security.

Integrity

Data is accurate and uncorrupted at all times

Consistency

All reads return the same committed state

Availability

Accessible when the system needs it

Security

Protected from unauthorized access and injection

SQL vs. NoSQL

There is no universally "best" database. The right choice depends on your data structure, scale, query patterns, and consistency requirements.

- **SQL:** PostgreSQL, MySQL, SQL Server
- **NoSQL:** MongoDB, Elasticsearch, Redis, Cassandra

i Choose based on the problem — not the trend.





When to Choose SQL

Rule of thumb: If data has clear relationships and correctness is non-negotiable, start with SQL.

Best Use Cases

- User management & authentication
- Orders, payments & billing
- Inventory & asset management

Key Characteristics

- Structured, enforced schema
- Foreign key relationships
- ACID transactions
- Strong consistency guarantees



When to Choose NoSQL

Rule of thumb: If data volume is massive or the schema evolves constantly, consider NoSQL.

Best Use Cases

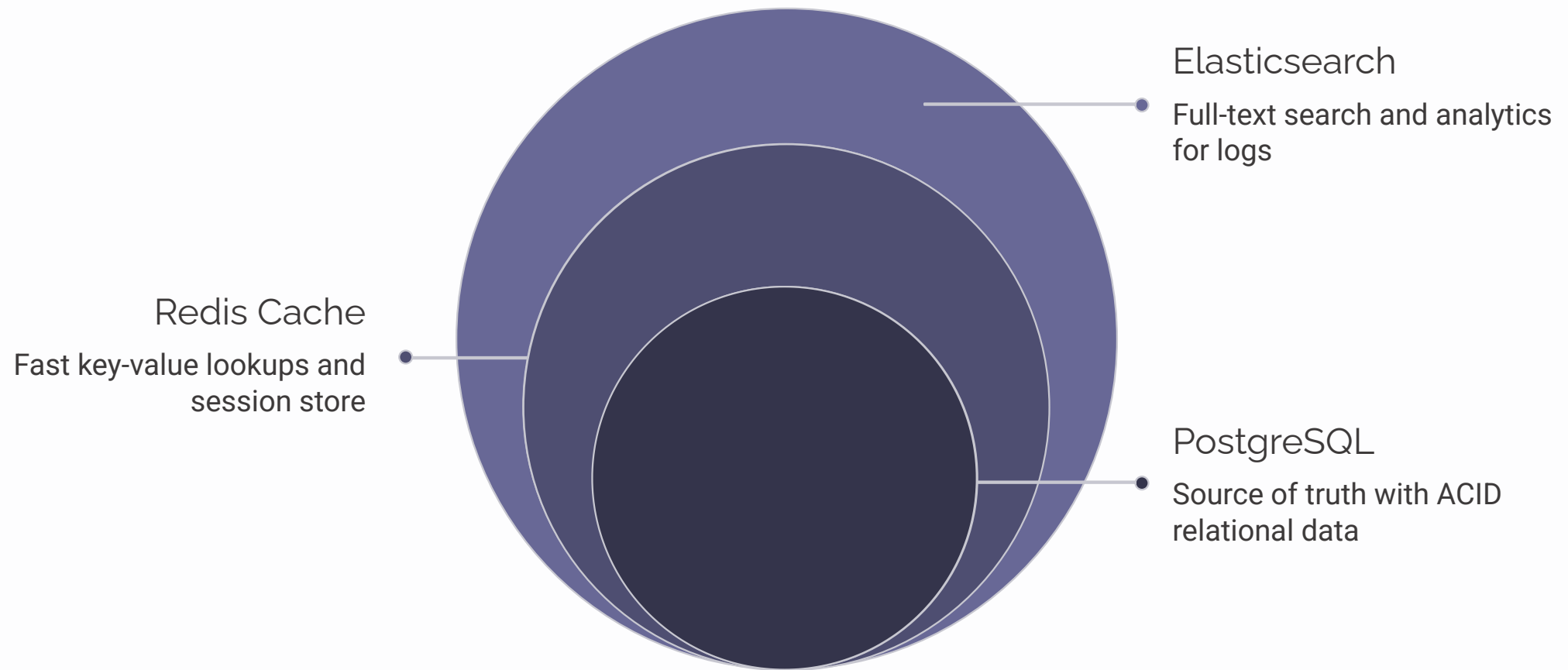
- Logs & security events
- Metrics & telemetry
- Search systems & caching

Key Characteristics

- Flexible, schema-less design
- High write throughput
- Horizontal scaling at scale
- Eventual consistency acceptable

Real-World Architecture

Production systems rarely rely on a single database. Each component is chosen for a specific job — together they form a purposeful, layered architecture.



Choose the right tool for each problem. Complexity is justified when it solves a real constraint.

Design From Business Entities

Start by asking: "**What business objects am I managing?**" — not "What tables should I create?"

For a Mini ASM security platform, entities evolve over time:

📄 Model the domain first. The schema follows naturally.

Current

Asset

Future

Asset · Finding · Scan · User

Asset Data Model

Core Fields

- `id` — UUID primary key
- `name` — asset identifier
- `type` — domain, ip, service
- `status` — active, archived
- `created_at` — timestamp

SQL Definition

```
CREATE TABLE assets (  
  id UUID PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  type VARCHAR(20) NOT NULL,  
  status VARCHAR(20) NOT NULL,  
  created_at TIMESTAMP NOT NULL  
);
```


Database Design Best Practices

01

Use stable primary keys

UUIDs over auto-increment integers for distributed systems

03

Enforce NOT NULL constraints

Eliminate ambiguity at the database level

02

Add created_at and updated_at

Every important table needs audit timestamps

04

Add CHECK constraints

CHECK (type IN ('domain', 'ip', 'service'))

Key Lessons From Production

- **Data outlives code** — design it carefully from day one
- **Most performance issues are query issues** — profile before optimizing
- **App validation is not enough** — enforce rules at the database layer
- **Every critical table needs timestamps** — for auditing and debugging
- **Backups are mandatory** — test restoration, not just backup creation

✅ **Key Takeaway:** Design data first. Technology comes second.

